

DEV_IMPL_GUIDE - SUB-WF-RESUME-01 Subscription Resume

1. Purpose

This guide explains resume implementation as the flow executes at runtime.

Use it with:

- DD_SUB-WF-RESUME-01-Subscription_Resume-v1.2.md
- DD_SUB-WF-RESUME-01-FLOW-WIK-v1.2.md
- DD_SUB-WF-FRAMEWORK-01-Subscription_Workflow_Framework-v1.0.md
- DD_SUB-WF-FRAMEWORK-01-WORKERS-v1.0.md

This guide is the parent-level implementation flow. Operator BPMN details are in the FLOW satellite.

2. Runtime Actors

Runtime component	What it does
subscription-workflow-api pod	Receives resume request, validates, starts operation.
SUB-LM module/API	Provides authoritative subscription status and mutation APIs.
Pause/history persistence	Provides the open pause-history row and closes it after success.
BIL-01	Computes resume intent, reconnection fee, and debt-cleared validation.
BIL-04	Calls DUNNING_RESUME after non-payment debt is settled.
FUL-03	Reactivates network service.
Camunda	Runs resume BPMN.
Drools	Validates trigger, role, origin-intent, and billing decisions.
Worker apps	Execute BPMN service task topics.
Scheduler	Calls SCHEDULED_AUTO_RESUME when resume_scheduled_at is due.

3. Before The Request Can Work

Required:

- subscription_resume_config seeded.
- subscription_pause_history exists and has resume close-out fields.
- subscription_operation_config has (operator_code=WIK, operation_kind=RESUME) -> sub-resume-wik.
- Drools package rules.subscription.resume.wik.
- BPMN process sub-resume-wik.
- Worker apps subscribe to required topics.
- FUL-03 can reactivate the service.
- BIL-01 supports RESUME_INTENT and RESUME_NP_INTENT.

4. Step 1 - Caller Sends Resume Request

Example dunning resume from BIL-04:

```
POST /api/subscriptions/sub_01HZ_KAMAU_FIBER/resume HTTP/1.1
Authorization: Bearer <bil04-service-token>
Idempotency-Key: resume-sub_01HZ_KAMAU-bil04-payment-cleared-20260622
Content-Type: application/json

{
  "operatorCode": "WIK",
  "resumeTrigger": "DUNNING_RESUME",
```

```

    "resumeReasonCode": "DEBT_CLEARED_VIA_MPESA",
    "dunningCycleReference": "bil04-resume-2026-06-22",
    "notes": "Customer paid INV_01HZ KES 3,500 via M-Pesa",
    "initiatingWorkflowKind": "BIL_04_DUNNING",
    "initiatingWorkflowRef": "bil04-resume-2026-06-22"
  }
}

```

Example scheduled auto-resume from scheduler:

```

POST /api/subscriptions/sub_01HZ_KAMAU_FIBER/resume HTTP/1.1
Authorization: Bearer <scheduler-service-token>
Idempotency-Key: auto-resume-pause_01HZ_KAMAU_VAC-20260415
Content-Type: application/json

{
  "operatorCode": "WIK",
  "resumeTrigger": "SCHEDULED_AUTO_RESUME",
  "resumeReasonCode": "SCHEDULED_AUTO_RESUME",
  "initiatingWorkflowKind": "FOUNDATION_SCHEDULER",
  "initiatingWorkflowRef": "pause_01HZ_KAMAU_VAC"
}

```

Actor user/role comes from auth context, not request body.

5. Step 2 - API Validates Shape And Idempotency

Validate request shape:

Field	Validation
operatorCode	Required.
resumeTrigger	One of CUSTOMER_REQUESTED_RESUME, ADMIN_FORCE_RESUME, DUNNING_RESUME, SCHEDULED_AUTO_RESUME.
resumeReasonCode	Required for audit.
dunningCycleReference	Required for DUNNING_RESUME.
adminForceResumeReason	Required for ADMIN_FORCE_RESUME on SUSPEND_NP.

Idempotency behavior:

- Same key and same request hash returns existing operation response.
- Same key and different request hash returns 409 IDEMPOTENCY_KEY_REUSED.
- Check existing key and hash before inserting a new operation.
- Use a DB unique constraint or transaction guard so concurrent duplicate calls cannot create two operations.

6. Step 3 - API Loads Required Data

6.1 Subscription Summary

Load from SUB-LM source of truth:

Same app:

```
SubscriptionReadService.getSummary(subscriptionId)
```

Split service:

```

GET /internal/subscriptions/sub_01HZ_KAMAU_FIBER/summary
Authorization: Bearer <subscription-workflow-service-token>

```

Expected:

```

{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "operatorCode": "WIK",
  "customerId": "cust_01HZ_KAMAU",
  "statusCode": "SUSPENDED",
  "packageRef": "pkg_FIBER_HOME_50M",
  "homepassId": "hp_01HZ_KAREN_017",
  "activeRestrictions": []
}

```

Do not use cache for current subscription status.

6.2 Resume Config

```
SELECT *
FROM subscription_resume_config
WHERE operator_code = 'WIK';
```

Expected:

```
{
  "operatorCode": "WIK",
  "customerSelfServiceEnabled": true,
  "reconnectionFeeOnCustomerResume": false,
  "dunningResumeVerifyDebtCleared": true,
  "customerNotificationEnabled": true
}
```

6.3 Open Pause-History Row

```
SELECT *
FROM subscription_pause_history
WHERE subscription_id = :subscriptionId
AND actual_resume_at IS NULL;
```

Expected dunning suspension row:

```
{
  "pauseId": "pause_01HZ_KAMAU_SUSPNP",
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "originIntent": "SUSPEND_NP",
  "systemManaged": true,
  "suspendedAt": "2026-06-15T02:00:18Z",
  "actualResumeAt": null,
  "dunningCycleReference": "bil04-dunning-wave-2026-06-15-L3",
  "outstandingDebtAmount": 3500.00,
  "outstandingDebtCurrency": "KES"
}
```

If no row exists, return 409 NO_OPEN_PAUSE_HISTORY.

7. Step 4 - API Applies Fast Rejections

Before calling the framework:

Check	Failure
Subscription status is SUSPENDED.	409 INVALID_STATE_FOR_RESUME
No other state operation in progress.	409 OPERATION_IN_PROGRESS
Actor role matches trigger.	403 ROLE_NOT_AUTHORIZED_FOR_TRIGGER
Open row exists.	409 NO_OPEN_PAUSE_HISTORY
Trigger matches open row origin_intent.	403 error from origin-intent matrix
Scheduler trigger has resume_scheduled_at.	400 NO_SCHEDULED_RESUME_TIME

This fast validation improves API response quality. The BPMN still re-checks the same rules with Drools after it starts.

8. Step 5 - API Calls Drools

Package:

```
rules.subscription.resume.wik
```

Facts:

```
{
  "operatorCode": "WIK",
  "subscription": {
    "subscriptionId": "sub_01HZ_KAMAU_FIBER",
    "statusCode": "SUSPENDED"
  },
  "request": {
    "resumeTrigger": "DUNNING_RESUME",
    "resumeReasonCode": "DEBT_CLEARED_VIA_MPESA",
    "dunningCycleReference": "bil04-resume-2026-06-22"
  },
  "openPause": {
    "pauseId": "pause_01HZ_KAMAU_SUSPNP",
    "originIntent": "SUSPEND_NP",
  }
}
```

```

        "systemManaged": true,
        "resumeScheduledAt": null,
        "actualResumeAt": null
    },
    "config": {
        "customerSelfServiceEnabled": true,
        "reconnectionFeeOnCustomerResume": false,
        "dunningResumeVerifyDebtCleared": true,
        "customerNotificationEnabled": true
    },
    "actor": {
        "userId": "bil04-svc-account",
        "role": "BILLING_INTERNAL"
    }
}

```

Expected decision:

```

{
    "eligible": true,
    "billingIntentKind": "RESUME_NP_INTENT",
    "requiresReconnectionFeePayment": false,
    "requiresDebtClearedCheck": true,
    "adminForceResume": false
}

```

9. Drools Rules To Configure

9.1 Subscription Must Be SUSPENDED

```

rule "resume requires suspended subscription"
when
    $req: ResumeRequestFact()
    $sub: SubscriptionStateFact(subscriptionId == $req.subscriptionId, statusCode != "SUSPENDED")
then
    modify($req) {
        setEligible(false);
        setEligibilityReasonCode("INVALID_STATE_FOR_RESUME")
    }
end

```

9.2 Open Pause Row Required

```

rule "resume requires open pause history"
when
    $req: ResumeRequestFact(eligible == null)
    not (PauseHistoryFact(subscriptionId == $req.subscriptionId, actualResumeAt == null))
then
    modify($req) {
        setEligible(false);
        setEligibilityReasonCode("NO_OPEN_PAUSE_HISTORY")
    }
end

```

9.3 Trigger Role Gating

```

rule "customer requested resume allowed roles"
when
    $req: ResumeRequestFact(
        resumeTrigger == "CUSTOMER_REQUESTED_RESUME",
        eligible == null,
        initiatingActorRole not in ("CUSTOMER_SELF_SERVICE", "CALL_CENTER", "SUBSCRIPTION_OPERATOR")
    )
then
    modify($req) {
        setEligible(false);
        setEligibilityReasonCode("ROLE_NOT_AUTHORIZED_FOR_TRIGGER")
    }
end

rule "dunning resume requires billing internal"
when
    $req: ResumeRequestFact(
        resumeTrigger == "DUNNING_RESUME",
        eligible == null,
        initiatingActorRole != "BILLING_INTERNAL"
    )
then
    modify($req) {
        setEligible(false);
    }
end

```

```

        setEligibilityReasonCode("ROLE_NOT_AUTHORIZED_FOR_TRIGGER")
    }
end

rule "scheduled auto resume requires scheduler internal"
when
    $req: ResumeRequestFact(
        resumeTrigger == "SCHEDULED_AUTO_RESUME",
        eligible == null,
        initiatingActorRole != "SCHEDULER_INTERNAL"
    )
then
    modify($req) {
        setEligible(false);
        setEligibilityReasonCode("ROLE_NOT_AUTHORIZED_FOR_TRIGGER")
    }
end

```

9.4 Origin-Intent Gating

```

rule "customer cannot resume admin pause"
when
    $req: ResumeRequestFact(resumeTrigger == "CUSTOMER_REQUESTED_RESUME", eligible == null)
    $pause: PauseHistoryFact(subscriptionId == $req.subscriptionId, actualResumeAt == null, originIntent == "ADMIN_PAUSE")
then
    modify($req) {
        setEligible(false);
        setEligibilityReasonCode("ADMIN_PAUSE_BLOCKS_CUSTOMER_RESUME")
    }
end

rule "customer cannot resume non payment suspension"
when
    $req: ResumeRequestFact(resumeTrigger == "CUSTOMER_REQUESTED_RESUME", eligible == null)
    $pause: PauseHistoryFact(subscriptionId == $req.subscriptionId, actualResumeAt == null, originIntent == "SUSPEND_NP")
then
    modify($req) {
        setEligible(false);
        setEligibilityReasonCode("SYSTEM_MANAGED_SUSPENSION")
    }
end

rule "dunning resume only resumes non payment suspension"
when
    $req: ResumeRequestFact(resumeTrigger == "DUNNING_RESUME", eligible == null)
    $pause: PauseHistoryFact(subscriptionId == $req.subscriptionId, actualResumeAt == null, originIntent != "SUSPEND_NP")
then
    modify($req) {
        setEligible(false);
        setEligibilityReasonCode("NOT_A_DUNNING_SUSPENSION")
    }
end

rule "scheduled auto resume requires scheduled time"
when
    $req: ResumeRequestFact(resumeTrigger == "SCHEDULED_AUTO_RESUME", eligible == null)
    $pause: PauseHistoryFact(subscriptionId == $req.subscriptionId, actualResumeAt == null, resumeScheduledAt == null)
then
    modify($req) {
        setEligible(false);
        setEligibilityReasonCode("NO_SCHEDULED_RESUME_TIME")
    }
end

```

9.5 Billing Intent Decision

```

rule "dunning resume uses resume non payment intent"
when
    $req: ResumeRequestFact(resumeTrigger == "DUNNING_RESUME")
    $cfg: ResumeConfigFact(dunningResumeVerifyDebtCleared == true)
then
    modify($req) {
        setBillingIntentKind("RESUME_NP_INTENT");
        setRequiresDebtClearedCheck(true);
        setRequiresReconnectionFeePayment(false)
    }
end

rule "customer resume reconnection fee"
when
    $req: ResumeRequestFact(resumeTrigger == "CUSTOMER_REQUESTED_RESUME")
    $cfg: ResumeConfigFact(reconnectionFeeOnCustomerResume == true)

```

```

then
  modify($req) {
    setBillingIntentKind("RESUME_INTENT");
    setRequiresReconnectionFeePayment(true)
  }
end

rule "resume billing default"
salience -100
when
  $req: ResumeRequestFact(billingIntentKind == null)
then
  modify($req) {
    setBillingIntentKind("RESUME_INTENT");
    setRequiresReconnectionFeePayment(false)
  }
end

```

10. Step 6 - API Starts Operation

After API validation and Drools approval, the resume API asks the shared SUB-WF framework to create the operation and start Camunda.

Same deployable:

```
OperationStartService.start(StartOperationCommand command)
```

Split service:

```

POST /internal/subscription-operations/start
Authorization: Bearer <subscription-workflow-service-token>
Content-Type: application/json

```

The payload below is the internal command passed to the framework start contract. It is not the public resume API request.

```

{
  "operatorCode": "WIK",
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "operationKind": "RESUME",
  "idempotencyKey": "resume-sub_01HZ_KAMAU-bil04-payment-cleared-20260622",
  "initiatingActorUserId": "bil04-svc-account",
  "initiatingActorRole": "BILLING_INTERNAL",
  "initiatingWorkflowKind": "BIL_04_DUNNING",
  "initiatingWorkflowRef": "bil04-resume-2026-06-22",
  "bpmnVariables": {
    "customerId": "cust_01HZ_KAMAU",
    "resumeTrigger": "DUNNING_RESUME",
    "resumeReasonCode": "DEBT_CLEARED_VIA_MPESA",
    "openPauseHistoryRowId": "pause_01HZ_KAMAU_SUSPNP",
    "openPauseOriginIntent": "SUSPEND_NP",
    "billingIntentKind": "RESUME_NP_INTENT",
    "requiresDebtClearedCheck": true,
    "requiresReconnectionFeePayment": false,
    "dunningCycleReference": "bil04-resume-2026-06-22",
    "notes": "Customer paid INV_01HZ KES 3,500 via M-Pesa"
  }
}

```

Framework starts Camunda process:

```
sub-resume-wik
```

11. Step 7 - API Returns Accepted

```

{
  "operationId": "subop_01HZ_KAMAU_RESUME",
  "operationKind": "RESUME",
  "bpmnProcessKey": "sub-resume-wik",
  "currentState": "INITIATED"
}

```

12. Step 8 - BPMN Executes Worker Topics

Order	BPMN service task does this	Worker topic/message
1	Re-check policy.	drools-decide
2	Put subscription in pending resume.	sub-lm-put-pending-status
3	Compute cycle window.	sub-lm-compute-cycle-window

Order	BPMN service task does this	Worker topic/message
4	Ask billing for resume intent.	bil01-emit-intent
5	Optional pay-first reconnection fee.	bil01-charge-pre-authorize, payment-confirmed, bil01-confirm-charge
6	Reactivate service.	ful-call-activate
7	Wait for fulfillment result.	fulfillment-completed or fulfillment-failed
8	Close pause-history row.	sub-lm-put-master-fields
9	Commit subscription active.	sub-lm-commit-state
10	Emit resume event.	kafka-emit
11	Notify customer.	notification-send
12	Finalize operation.	subscription-operation-finalize

These names are worker topics or BPMN message names, not shell commands.

13. Step 9 - Billing Intent Details

For dunning resume:

```
{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "intentKind": "RESUME_NP_INTENT",
  "intentContext": {
    "resumeTrigger": "DUNNING_RESUME",
    "openPauseOriginIntent": "SUSPEND_NP",
    "dunningCycleReference": "bil04-resume-2026-06-22",
    "verifyDebtCleared": true
  }
}
```

Expected success:

```
{
  "intentDecisionId": "bidec_01HZ_KAMAU_RESUME",
  "debtCleared": true,
  "lineItems": [],
  "totals": {
    "net": 0,
    "currency": "KES"
  }
}
```

If BIL-01 returns debtCleared=false, BPMN emits SubscriptionResumeRejected and leaves subscription SUSPENDED.

14. Step 10 - Optional Payment Gate

The payment gate runs only when:

```
requiresReconnectionFeePayment == true
```

Then BPMN executes:

```
bil01-charge-pre-authorize
wait payment-confirmed with boundary timer P7D
bil01-confirm-charge
```

For WIK v1.2 default config, this is normally skipped because:

```
reconnection_fee_on_customer_resume = false
```

15. Step 11 - FUL-03 Reactivates Service

Service task topic:

```
ful-call-activate
```

Worker input:

```
{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "intent": "RESUME_SERVICE",
}
```

```

    "resumeTrigger": "DUNNING_RESUME",
    "fulfillmentCorrelationId": "subop_01HZ_KAMAU_RESUME:resume"
  }

```

BPMN waits for:

```

    fulfillment-completed
    fulfillment-failed

```

Correlate on:

```

    fulfillmentCorrelationId = subop_01HZ_KAMAU_RESUME:resume

```

16. Step 12 - BPMN Closes Pause-History Row

Service task topic:

```

    sub-lm-put-master-fields

```

Worker input:

```

{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "fieldsJson": {
    "last_status_changed_at": "2026-06-22T10:00:18Z"
  },
  "pauseHistoryCloseOut": {
    "pauseId": "pause_01HZ_KAMAU_SUSPNP",
    "actualResumeAt": "2026-06-22T10:00:18Z",
    "resumeReasonCode": "DEBT_CLEARED_VIA_MPESA",
    "resumeActorUserId": "bil04-svc-account",
    "resumeActorRole": "BILLING_INTERNAL",
    "reconnectionFeeBillableEventId": null,
    "dunningCycleReferenceAtResume": "bil04-resume-2026-06-22",
    "adminForceResume": false
  }
}

```

Implementation guidance:

- If SUB-LM and pause-history persistence are in one module, write status audit fields and pause-history close-out in one transaction.
- If they are split, call approved internal APIs or an orchestration service.
- Do not close the row before FUL-03 reactivation succeeds.
- Do not close the row if final state commit cannot be completed; route to manual recovery if needed.

17. Step 13 - BPMN Commits ACTIVE

Service task topic:

```

    sub-lm-commit-state

```

Worker input:

```

{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "expectedFromStatus": "PENDING_RESUME",
  "toStatus": "ACTIVE"
}

```

Only after this succeeds can the BPMN emit SubscriptionResumed.

18. Step 14 - Success Event Is Emitted

Event:

```

    SubscriptionResumed

```

Topic:

```

    sophix.subscription.subscription.resumed

```

Emitted only after SUB-LM commits ACTIVE.

19. Compensation Flows

Failure	BPMN compensation
Validation reject before pending	kafka-emit SubscriptionResumeRejected, finalize failed.
Billing debt not cleared	kafka-emit SubscriptionResumeRejected, finalize failed; subscription stays SUSPENDED.
Reconnection fee payment timeout	bil01-revert-pending-charge, sub-lm-revert-to-prior, emit rejected, finalize failed.
FUL-03 failed	bil01-revert-pending-charge if needed, sub-lm-revert-to-prior, emit rejected, finalize failed.
Pause-history close-out failed after network activation	Manual recovery.
Commit ACTIVE failed after network activation	Manual recovery.
Notification failed	Log and continue.

20. Manual Smoke Test - Dunning Resume

1. Seed subscription_resume_config.
2. Seed operation config (WIK, RESUME) -> sub-resume-wik.
3. Create a suspended subscription.
4. Create open subscription_pause_history row with origin_intent = SUSPEND_NP.
5. Send resume API request with DUNNING_RESUME using BIL-04 service token.
6. Confirm operation row exists.
7. Confirm BPMN process exists.
8. Confirm subscription moves to PENDING_RESUME.
9. Confirm BIL-01 receives RESUME_NP_INTENT.
10. Simulate or receive FUL-03 fulfillment-completed.
11. Confirm pause-history row has actual_resume_at populated.
12. Confirm subscription becomes ACTIVE.
13. Confirm SubscriptionResumed outbox row exists.
14. Confirm operation final state is COMPLETED.

21. Manual Smoke Test - Scheduled Auto Resume

1. Create a suspended subscription with open pause row:
origin_intent = CUSTOMER_REQUESTED_PAUSE, resume_scheduled_at <= now, actual_resume_at = NULL.
2. Call resume API with scheduler service token and SCHEDULED_AUTO_RESUME.
3. Confirm origin-intent gate accepts the request.
4. Confirm BPMN skips reconnection fee for WIK default config.
5. Confirm FUL-03 reactivates service.
6. Confirm pause-history row closes.
7. Confirm subscription becomes ACTIVE.

22. Common Mistakes

- Reading current subscription status from cache.
- Starting BPMN without loading the open pause-history row.
- Letting customers self-resume SUSPEND_NP.
- Letting BIL-04 dunning-resume voluntary pauses.
- Closing pause history before FUL-03 succeeds.

- Emitting SubscriptionResumed before sub-lm-commit-state.
- Treating worker-topic names as shell commands.
- Creating separate worker deployments for every topic without an operations need.
- Failing the whole operation because NOT-01 notification failed.